

# NumPy 数组与数值计算 练习解答

本文件对应第五讲的 14 道练习。先独立写出数值、形状和理由，再用这里的推导核对。一个数字相同不代表实现相同，尤其应留意是否保留了样本轴和是否发生了意外广播。

## 第一题 一维不是一列

`a.shape` 是 `(4,)`，`a.ndim` 是 `1`，`a.size` 是 `4`。形状元组只有一个长度，说明数组只有一条轴；`(4, 1)` 有两个长度，说明存在两条轴，第一条长 `4`、第二条长 `1`。两者元素总数相同，但后续广播行为不同。`a.reshape(4, 1)` 才会得到四行一列的二维数组。

不要用打印出来是不是竖着排列作为判据。显示宽度会改变排版，数组语义由形状决定。最可靠的核验是打印 `.shape`，而不是数屏幕上有几行字符。

## 第二题 整数索引与范围切片

`X[1, 0]` 是 NumPy 数值标量 `60.0`，形状为 `()`。`X[1:2]` 是 `[[60.0, 2.0]]`，形状为 `(1, 2)`。`X[:, 1]` 是 `[1.0, 2.0, 1.0]`，形状为 `(3,)`。`X[:, 1:2]` 是 `[[1.0], [2.0], [1.0]]`，形状为 `(3, 1)`。

把第二个样本交给批处理预测函数，应使用 `X[1:2]`。这样样本轴仍然存在，函数看到的是一批只有一个样本的数据。若写 `X[1]`，得到形状 `(2,)`，样本轴被去掉；严格的批处理接口应明确拒绝它。

## 第三题 模型 C 的完整计算

逐行预测为  $1.5 \times 50 + 40 = 115$ 、 $1.5 \times 60 + 40 = 130$ 、 $1.5 \times 80 + 40 = 160$ 、 $1.5 \times 90 + 40 = 175$ ，单位为万元。按“预测减标签”的约定，带符号误差为 `[5, 0, -10, -15]` 万元，绝对误差为 `[5, 0, 10, 15]` 万元。

绝对误差之和为 30 万元，除以四个样本，MAE 为 7.5 万元。数组表达式可以分四行写清楚：

```
prediction = 1.5 * area + 40.0
signed_error = prediction - target
absolute_error = np.abs(signed_error)
mae = absolute_error.mean()
```

仅比较 MAE 不够，因为 `[0, 20]` 与 `[10, 10]` 都有均值 10，却对应不同样本误差。应先比较预测和逐项误差的形状与数值，再比较最终平均数。

## 第四题 指定被合并的轴

`Z.sum(axis=0)` 把三个样本位置合并，得到 `[9, 12]`，形状 `(2,)`。`Z.sum(axis=1)` 把每行两个特征位置合并，得到 `[3, 7, 11]`，形状 `(3,)`。`Z.mean()` 不指定轴，合并所有六个元素，得到数值标量 `3.5`，形状 `()`。

`keepdims=True` 保留被合并轴的位置并让长度变成 1。因此 `Z.sum(axis=0, keepdims=True)` 为 `[[9, 12]]`，形状 `(1, 2)`；`Z.sum(axis=1, keepdims=True)` 为 `[[3], [7], [11]]`，形状 `(3, 1)`。它改变输出结构，不改变这些和的数值。

## 第五题 从最右轴开始比较

`(3, 2)` 与 `(2,)` 可广播。后者在比较时视为 `(1, 2)`，得到 `(3, 2)`。

`(3, 2)` 与 `(3,)` 不可广播。最右轴长度 2 与 3 不相等，也没有一个为 1。左侧是否有三个样本不能挽救最右轴的不兼容。

`(3, 2)` 与 `(3, 1)` 可广播。最右轴 2 与 1 兼容，左侧都是 3，结果 `(3, 2)`。

`(1, 2)` 与 `(2,)` 可广播，后者视为 `(1, 2)`，结果 `(1, 2)`。这正是单样本仍按每个特征乘对应系数的情况。

## 第六题 看似合理的错误均值

正确带符号误差是 `[10-12, 20-18] = [-2, 2]`，绝对误差 `[2, 2]`，MAE 为 2。

列状预测与一维标签相减后，第一行用 10 减所有标签，得到 `[-2, -8]`；第二行用 20 减所有标签，得到 `[8, 2]`。错误绝对差共有四个数，和为 20，错误 MAE 为 5。

形状由 `(2, 1)` 和 `(1, 2)` 广播成 `(2, 2)`，其中非对角位置在比较不同样本。修复接口应要求预测和标签都为 `(n,)`，并先检查形状相等。本题若只留下最终数字 5，很难看出错在哪里，因此调试应查看相减结果本身。

## 第七题 单样本仍然是一批

预测为  $70 \times 2 + 3 \times 5 + 10 = 165$  万元。构造输入时需要两层方括号，外层表示这批数据，内层表示唯一的样本：

```
single_X = np.array([[70.0, 3.0]])
single_prediction = weighted_predict(single_X, [2.0, 5.0], 10.0)
assert single_X.shape == (1, 2)
assert single_prediction.shape == (1,)
assert single_prediction[0] == 165.0
```

输出是 `(1,)`，因为函数约定每个样本一个预测，并返回一维结果。它不是标量，因为批处理接口不应随样本数变为 1 而突然更换返回形式；也不是 `(1, 1)`，因为本讲没有设置第二条输出轴。

## 第八题 拒绝而不是猜测

`X` 有两列，长度为 3 的系数不满足 `(d,)`，应抛出清楚的 `ValueError`。`X[0]` 是一维数组，不符合 `(n, d)`，也应拒绝。调用者若确实只预测第一个样本，应传入 `X[0:1]`。

压平会丢掉样本与特征边界；补齐系数则擅自改变模型；自动猜测输入究竟表示一套房的两个特征还是两套房的一个特征，也没有依据。接口可以设计便利转换，但必须明确规定并独立测试，不能在报错后临时凭数值凑形状。

## 第九题 共享内存的后果

修改后 `a` 是 `[10, 200, 30, 40]`，`b` 是 `[200, 30]`，`c` 仍是 `[20, 30]`。`b` 是基本切片返回的视图，`b[0]` 与 `a[1]` 对应同一处数据。`c` 在修改前已显式复制，所以保持旧值。

```
assert np.shares_memory(a, b)
assert not np.shares_memory(a, c)
```

这两个检查针对数组是否共享底层数据。`b is a` 是另一问题：两个变量是否指向同一个对象。一个切片可以是不同对象，却仍共享存储；不要把对象身份检查当作共享内存检查。

## 第十题 容器放不下正确答案

`int8` 最大只能表示 127，正确结果 130 超出范围。在本讲的固定宽度数组相加实验中发生溢出，得到 -126。金额没有改变含义，是数值格式无法容纳答案。先转换类型再计算：

```
small = np.array([120], dtype=np.int8)
safe = small.astype(np.int64) + 10
assert safe[0] == 130
```

若先做溢出运算，再把 -126 转为 `int64`，已经错掉的数不会恢复。把 1.9 写入整数数组会丢弃小数部分，保存为 1；若希望保留小数，构造或赋值前应使用浮点数组。`int64` 也有范围，并非可以表示任意大的整数。

## 第十一题 提高精度不能恢复历史信息

`float32` 在一亿附近不能区分每个相邻整数。`1000000001` 存入时会被舍入，之后数组实际含有的是 `1000000000`。转换为 `float64` 只会更精确地保存现有数值，无法知道原来末尾是否有 1。

本例可从原始整数或高精度来源直接构造：`np.array([100000001], dtype=np.float64)`。这里目标整数在 `float64` 可精确表示的整数范围内，因此不会出现本例的舍入。这个例子不表示所有整数都能被 `float64` 精确表示；更大的整数仍须另行检查范围与精度需求。

## 第十二题 平方增长带来的代价

正确误差有 10000 个元素。每个 `float64` 元素占 8 字节，因此结果数据占  $10000 \times 8 = 80000$  字节，即 80 kB 十进制字节单位。

错误交叉差有  $10000 \times 10000 = 100000000$  个元素，结果数据占 800000000 字节，即 800 MB 十进制字节单位。错误结果比正确结果多 10000 倍。若使用二进制单位，数值换算不同，不能把 MB 与 MiB 混为一谈。

这里没有计入输入和临时数组，也不是整个程序的峰值内存。实际运行大规模错误广播可能直接耗尽内存，因此用纸笔估算和小规模复现即可，不要为了验证本题真的创建巨型数组。

## 第十三题 计时结论需要范围

至少应追问：两个版本是否做同一批预测和同一种 MAE；有没有一方包含打印、加载或列表转数组，而另一方没有；样本数和特征数是多少；是否已经检查结果一致；是否重复测量并说明汇总方法；机器负载、NumPy 版本和数据类型是什么。

对一个规模的一次测量，不能推出所有规模的速度关系。小数组的固定调用开销可能使数组版本更慢，大数组还可能受内存带宽与临时分配影响。合理结论应该是“在这些输入、这套环境和这些计时边界下，本次重复测量得到某个中位时间”，而不是“永远快 100 倍”。

## 第十四题 一个小而明确的测试

以下代码假设已导入 NumPy 和实验中的 `weighted_predict`，正文为八行：

```
X = np.array([[50., 1.], [60., 2.], [80., 1.]])
before = X.copy()
p = weighted_predict(X, [2., 5.], 10.)
assert p.shape == (3,)
np.testing.assert_allclose(p, [115., 140., 175.], rtol=0, atol=1e-12)
one = weighted_predict(X[:1], [2., 5.], 10.)
assert one.shape == (1,) and one[0] == 115.
np.testing.assert_array_equal(X, before)
```

它验证特定正常输入的数值、形状与不修改输入行为，不能证明所有输入都正确，也没有检验空数组、非有限值、错误系数或真实预测效果。测试应有明确范围，不能把几次通过解释为无条件正确。